

Fixing ball game for SE lab

<https://gemini.google.com/app/0007ae7caac6f3a5>

User prompt: how to clone github repo?

Response: Cloning a GitHub repository creates a local copy of the repository on your computer.

Step 1: Get the Repository URL

1. Go to the repository page on **GitHub**.
2. Click the green **Code** button.
3. Select your preferred method (HTTPS is standard for beginners) and copy the URL.

Step 2: Open Terminal or Command Prompt

- **Windows:** Press Win + R, type cmd or powershell, and press Enter. (Or open **Git Bash** if installed).
- **Mac/Linux:** Open the **Terminal** app.

Step 3: Run the git clone Command

Navigate to the directory where you want to save the folder, then run:

```
git clone https://github.com/username/repository-name.git
```

Step 4: Open the Cloned Repository

Once complete, navigate into the downloaded folder:

```
cd repository-name
```

```
callout-content-12
```

User prompt: https://github.com/SETAPESU26/06_coin_collector what do I do now?

Response:

1. **Review the Requirements:** Read the ReadMe file in the 06_coin_collector repository to understand the required features.
2. **Fork and Clone:** Fork the repository so you have a copy in your personal account, then clone it to your computer.
3. **Run and Record (Before):** Execute the initial Python puzzle/game code and record a 10-second video of it running before making any modifications.
4. **Vibe Code the Fixes:** Write prompts to fix the broken code, then write additional prompts to add the features listed in the ReadMe one by one.

5. **Commit Progressively:** Make separate Git commits for each individual task you complete.
 6. **Record (After):** Record a final 10-second video demonstrating the fully functioning, updated code.
 7. **Push Your Changes:** Push the code to your personal repository. Do not raise a Pull Request back to the main SETAPESU26 repository.
 8. **Submit Deliverables:** Export this chat history as a DOC or PDF. Upload the chat document, both videos, and your updated code to your Lab repository inside the "Lab-4" folder.
-

User prompt: tell me how to fork and clone exactly, where to do what

Response: 1. Fork the Repository (Where: In your web browser)

Forking creates a copy of the original repository in your personal GitHub account so you can make changes without affecting the original.

1. Go to the original repository: https://github.com/SETAPESU26/06_coin_collector
2. In the top-right corner of the page, click the **Fork** button.
3. Under "Owner," make sure your personal GitHub username is selected.
4. Click the **Create fork** button.

You will now be redirected to a new page. Notice the URL has changed to https://github.com/YOUR-USERNAME/06_coin_collector. This is your personal copy.

2. Clone Your Fork (Where: In your local terminal/command line)

Cloning downloads your personal copy to your computer so you can edit the files.

1. On your newly forked repository page, click the green **Code** button.
2. Under the **Local** tab, copy the HTTPS web URL (it will look like https://github.com/YOUR-USERNAME/06_coin_collector.git).
3. Open your Terminal (Mac/Linux) or Command Prompt / Git Bash (Windows).
4. Navigate to the folder where you want to save your project using the `cd` command. For example:

```
cd Documents/Labs
```

5. Run the `git clone` command followed by the URL you copied:

```
git clone https://github.com/YOUR-USERNAME/06_coin_collector.git
```

6. Once the download finishes, move into the new project folder:

```
cd 06_coin_collector
```

You are now ready to run the game, record your before video, and start using your Vibe Coding tool to fix the code.

User prompt: what about this answer in 20 words

Response:

User prompt: what about this answer in 20 words

Response: Keep it checked. It copies only the main branch to your fork, which is all you need.

User prompt: ok now I have forked and clone what are next exact steps

Response: Here are your exact next steps based on the lab instructions:

- **Navigate into your project folder:** Open your terminal and run `cd 06_coin_collector`.
 - **Read the instructions:** Open and read the `README.md` file inside the folder to see the deliverables and required features.
 - **Run the code:** Execute the Python script to test the initial game.
 - **Record "before" video:** Record a
-

User prompt: ok now I have forked and clone what are next exact steps

Response:

1. **Navigate into the folder:** Open your terminal and run `cd 06_coin_collector`.
2. **Read the instructions:** Open the `README.md` file to understand the deliverables and the specific features you need to add.
3. **Run the code:** Execute the Python game/puzzle code.
4. **Record your baseline:** Record a 10-second video of the code running *before* you make any changes.
5. **Fix the code:** Use your Vibe Coding tool (e.g., ChatGPT, Gemini, Claude) to write prompts that fix the broken code.
6. **Add features progressively:** Write prompts to add the features listed in the ReadMe file one by one. You must make a separate Git commit for each task.
7. **Record your final result:** Record another 10-second video showing the code running after making all the changes.

8. **Push to your fork:** Push the updated code back to your personal repository. Do not raise a Pull Request to the main SETAPESU26 repository.
 9. **Submit deliverables:** Export this chat history as a document or PDF, and upload it along with both videos and the updated code to your Lab repository under the "Lab-4" folder.
-

User prompt: ok now I have forked and clone what are next exact steps

Response:

1. **Navigate into the folder:** In your terminal, run `cd 06_coin_collector`.
 2. **Review the requirements:** Open the `README.md` file in that folder to understand the exact deliverables and features you need to add.
 3. **Run the initial code:** Execute the Python file (e.g., `python main.py` or whatever the main script is named) to launch the broken game.
 4. **Record the baseline:** Screen-record a 10-second video of the game running in its current, broken state before you change anything.
 5. **Start Vibe Coding:** Use your AI tool to write prompts to fix the broken code.
 6. **Add features and commit:** Prompt the AI to add the features listed in the README one by one. You must make a separate Git commit for every single task (e.g., `git add .` then `git commit -m "Fix character movement"`).
-

User prompt: ok now I have forked and clone what are next exact steps?

Response:

1. Navigate into the cloned folder using your terminal by running `cd 06_coin_collector`.
2. Read the ReadMe file to understand the specific deliverables and features you need to add.
3. Run the Python-based game or puzzle code.
4. Record 10 seconds of video demonstrating the code before making any changes.
5. Use your Vibe Coding tool to write prompts and fix the broken code.
6. Write prompts to add all the features listed in the ReadMe file one by one.
7. Make separate Git commits for each task you complete.
8. Record another 10 seconds of video after making all the changes to show the final result.
9. Push the updated code to your personal repository. Do not raise a Pull Request to the main repo (SETAPESU26).
10. Export your AI chat history as a doc or pdf.

11. Upload the before and after videos, the updated code, and the chat history to your Lab repository under the Lab-4 folder.
-

User prompt: how do I run the code and play it and record? tell me step by step

Response: 1. Find and Run the Python File

1. Open your terminal and ensure you are inside the cloned folder (run `cd 06_coin_collector`).
2. List the files in the folder to find the main Python script by running `dir` (Windows) or `ls` (Mac/Linux). Look for a file like `main.py`, `game.py`, or `coin_collector.py`.
3. Run the code by typing `python filename.py` (replace `filename.py` with the actual name of the file) and pressing Enter. If that doesn't work, try `python3 filename.py`.
4. The game window should now pop up on your screen.

2. Set Up Your Screen Recorder

You don't need special software; use your computer's built-in tools:

- **Windows:** Press `Windows Key + G` to open the Xbox Game Bar, then click the **Record** circle icon. Alternatively, open the **Snipping Tool**, click the video camera icon, and select your game window.
- **Mac:** Press `Command + Shift + 5`. Select "Record Selected Portion," highlight the game window, and click **Record**.

3. Record the "Before" Video

1. With the game running and the screen recorder active, try to play the game or let it run in its broken state.
2. Record exactly 10 seconds of this baseline behavior.
3. Stop the recording and save the file somewhere easy to find (e.g., your Desktop), naming it something like `before_video.mp4`.

Once this is saved, you can close the game and begin using your AI tool to generate prompts to fix the code.

User prompt: [notice] A new release of pip is available: 25.3 -> 26.2.1 [notice] To update, run: python.exe -m pip install --upgrade pip ERROR: Failed to build 'pygame' when getting requirements to build wheel PS C:\Users\Anirudh\Downloads\06_coin_collector\coin-collector> python main.py Traceback (most recent call last): File "C:\Users\Anirudh\Downloads\06_coin_collector\coin-collector\main.py", line 9, in <module> import pygame ModuleNotFoundError: No module named 'pygame' PS C:\Users\Anirudh\Downloads\06_coin_collector\coin-collector> what do i do

Response:

1. **Update pip:** Your terminal notice indicates your pip version is outdated, which is likely causing the pygame wheel build to fail. Run this command to upgrade it:

```
python -m pip install --upgrade pip
```

2. **Install pygame:** Once pip is updated, attempt to install the required library again:

```
python -m pip install pygame
```

3. **Run your code:** If the installation is successful, run your game script:

```
python main.py
```

Alternative Fix: If pygame still throws a wheel build error after upgrading pip, install Pygame Community Edition (pygame-ce). It is fully compatible with standard Pygame but has better support for newer Python versions:

```
python -m pip install pygame-ce
```

```
User prompt: PS C:\Users\Anirudh\Downloads\06_coin_collector\coin-collector> python -m pip
install pygame      Collecting pygame  Using cached pygame-2.6.1.tar.gz (14.8 MB)  Installing
build dependencies ... done  Getting requirements to build wheel ... error  error: subprocess-
exited-with-error    × Getting requirements to build wheel did not run successfully. | exit code:
1  ↳ [112 lines of output]    Skipping Cython compilation          WARNING, No
"Setup" File Exists, Running "buildconfig/config.py"    Using WINDOWS configuration...
  Making dir :prebuilt_downloads:    Downloading... https://www.libsdl.org/release/SDL2-devel-
2.28.4-VC.zip 25ef9d201ce3fd5f976c37dddedac36bd173975c    Unzipping
:prebuilt_downloads\SDL2-devel-2.28.4-VC.zip:    Downloading...
https://www.libsdl.org/projects/SDL_image/release/SDL2_image-devel-2.0.5-VC.zip
137f86474691f4e12e76e07d58d5920c8d844d5b    Unzipping
:prebuilt_downloads\SDL2_image-devel-2.0.5-VC.zip:    Downloading...
https://github.com/libsdl-org/SDL_ttf/releases/download/release-2.20.1/SDL2_ttf-devel-2.20.1-
VC.zip 371606aceba450384428fd2852f73d2f6290b136    Unzipping
:prebuilt_downloads\SDL2_ttf-devel-2.20.1-VC.zip:    Downloading... https://github.com/libsdl-
org/SDL_mixer/releases/download/release-2.6.2/SDL2_mixer-devel-2.6.2-VC.zip
000e3ea8a50261d46dbd200fb450b93c59ed4482    Unzipping
:prebuilt_downloads\SDL2_mixer-devel-2.6.2-VC.zip:    Downloading...
https://github.com/pygame/pygame/releases/download/2.1.3.dev4/prebuilt-x64-pygame-2.1.4-
20220319.zip 16b46596744ce9ef80e7e40fa72ddbafef1cf586    Unzipping
:prebuilt_downloads\prebuilt-x64-pygame-2.1.4-20220319.zip:    copying into .\prebuilt-x64
Path for SDL: prebuilt-x64\SDL2-2.28.4    ...Library directory for SDL: prebuilt-x64\SDL2-
2.28.4/lib/x64    ...Include directory for SDL: prebuilt-x64\SDL2-2.28.4/include    Path for
FONT: prebuilt-x64\SDL2_ttf-2.20.1    ...Library directory for FONT: prebuilt-x64\SDL2_ttf-
2.20.1/lib/x64    ...Include directory for FONT: prebuilt-x64\SDL2_ttf-2.20.1/include    Path for
IMAGE: prebuilt-x64\SDL2_image-2.0.5    ...Library directory for IMAGE: prebuilt-
x64\SDL2_image-2.0.5/lib/x64    ...Include directory for IMAGE: prebuilt-x64\SDL2_image-
2.0.5/include    Path for MIXER: prebuilt-x64\SDL2_mixer-2.6.2    ...Library directory for
MIXER: prebuilt-x64\SDL2_mixer-2.6.2/lib/x64    ...Include directory for MIXER: prebuilt-
x64\SDL2_mixer-2.6.2/include    Path for PORTMIDI: prebuilt-x64    ...Library directory for
PORTMIDI: prebuilt-x64/lib    ...Include directory for PORTMIDI: prebuilt-x64/include    DLL
for SDL2: prebuilt-x64\SDL2-2.28.4/lib/x64/SDL2.dll    DLL for SDL2_ttf: prebuilt-x64\SDL2_ttf-
2.20.1/lib/x64/SDL2_ttf.dll    DLL for SDL2_image: prebuilt-x64\SDL2_image-
2.0.5/lib/x64/SDL2_image.dll    DLL for SDL2_mixer: prebuilt-x64\SDL2_mixer-
2.6.2/lib/x64/SDL2_mixer.dll    DLL for portmidi: prebuilt-x64/lib/portmidi.dll    Path for
```

7/10

```

in setup_prebuilt_sdl2      DEPS.configure()      ~~~~~^^      File
"C:\Users\Anirudh\AppData\Local\Temp\pip-install-
hm2wtsth\pygame_771c20e2fcb0446aa0f25449fa615ab4\buildconfig\config_win.py", line 338,
in configure      from buildconfig import vstools      File
"C:\Users\Anirudh\AppData\Local\Temp\pip-install-
hm2wtsth\pygame_771c20e2fcb0446aa0f25449fa615ab4\buildconfig\vstools.py", line 6, in
<module>      from setuptools._distutils.msvccompiler import MSVCCompiler,
get_build_architecture      ModuleNotFoundError: No module named
'setuptools._distutils.msvccompiler'      [end of output]      note: This error originates from a
subprocess, and is likely not a problem with pip. ERROR: Failed to build 'pygame' when getting
requirements to build wheel PS C:\Users\Anirudh\Downloads\06_coin_collector\coin-collector>
python main.py      Traceback (most recent call last):      File
"C:\Users\Anirudh\Downloads\06_coin_collector\coin-collector\main.py", line 9, in <module>
import pygame ModuleNotFoundError: No module named 'pygame' PS
C:\Users\Anirudh\Downloads\06_coin_collector\coin-collector>

```

Response: The issue is that you are running **Python 3.14** (.../Python314/...), which removed standard support for legacy `distutils` modules that standard Pygame needs during source compilation. Because pre-built binary wheels for standard Pygame aren't built for Python 3.14 yet, `pip` tries to compile from source and fails.

Run the following solutions in order in your terminal:

Solution 1: Install pygame-ce (Recommended)

`pygame-ce` (Community Edition) is actively updated and provides pre-built wheels for newer Python versions. It is a drop-in replacement for standard Pygame:

```
python -m pip install pygame-ce
```

Once installed, test running your game:

```
python main.py
```

Solution 2: Install setuptools to fix distutils

If you specifically need standard `pygame`, install the latest `setuptools` package first (which re-adds missing build tools) and install Pygame with pre-releases allowed:

```
python -m pip install setuptools
python -m pip install pygame --pre
```

Solution 3: Use Python 3.11 or 3.12 (If above options fail)

If your Python 3.14 environment continues to block Pygame installations:

1. Download and install **Python 3.12** from python.org.
2. Run the file explicitly with Python 3.12:

```
py -3.12 -m pip install pygame
py -3.12 main.py
```


User prompt: Ok I ran the game recorded playing it, whats next?

Response:

1. **Fix the broken code:** Open your preferred AI tool (like ChatGPT, Gemini, or Claude) and prompt it with the error messages or bugs you noticed while running the game to fix the core issues.
2. **Add features one by one:** Look at the README .md file in the repository for the list of required features. Prompt the AI to help you build each feature individually.
3. **Commit after every task:** After completing and verifying each fix or feature, save the changes in Git with a descriptive commit message. For example:

```
git add .  
git commit -m "Fix character movement bug"
```

4. **Record the "after" video:** Once all features are implemented and working, screen-record a 10-second video of the updated, fully functioning game.
5. **Push to your fork:** Upload your completed commits to your personal GitHub repository:

```
git push origin main
```

6. **Export your chat:** Download or export this entire AI chat history as a PDF or Word document.
 7. **Submit deliverables:** Move the "before" video, "after" video, updated code, and chat export file into the Lab-4 folder of your personal Lab repository. Do not open a Pull Request to the main SETAPESU26 repository.
-

User prompt: so each change has to be added individual?

Response: Yes, exactly. The lab instructions require you to have separate commits for each task or feature you complete.

Follow this workflow for every change:

1. **Prompt the AI** to fix one bug or implement one specific feature from the README.
2. **Apply the code change** and test it in your game to make sure it works.
3. **Stage and commit the specific change** in your terminal:

```
git add .  
git commit -m "Add core score counter feature"
```

4. **Repeat** for the next feature or fix.

Once all required features are done and committed separately, you can push everything to your repository using `git push origin main`.

User prompt: ok i have made all commits and have recorded vids, what do i do?

Response:

1. **Push your code to GitHub:** Open your terminal and push all your commits to your personal forked repository:

```
git push origin main
```

(Ensure you **do not** create a Pull Request to the main SETAPESU26 repository.)

2. **Export your chat history:** Download or export this entire AI chat conversation as a .doc or .pdf file.
 3. **Organize your deliverables:** Place the following files inside your main personal Lab repository under the Lab-4 folder:
 - Before video (10 seconds)
 - After video (10 seconds)
 - Updated project code
 - Chat history export (.doc or .pdf)
 4. **Commit and push your lab repository:** Commit the added files and push your main Lab repo to GitHub to finish your submission.
-